THE NATIONAL UNIVERSITY OF LESOTHO

Pipeline and Hazard-Unit Lead: Architecture and Data Model

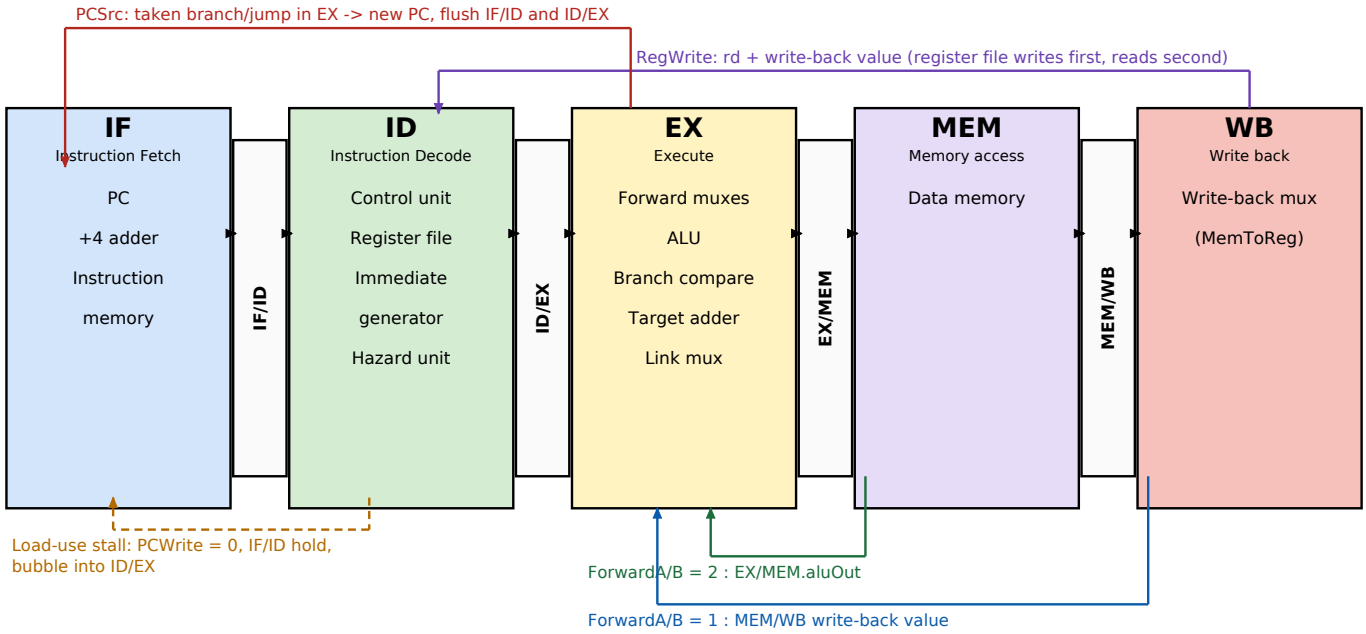
Role document for Milestone M2 · Tumelo Bernard Leputla

Team	limo-nul (lab group C)	**Release tag**	m2
Role	**Pipeline and hazard-unit lead**	**Member**	Tumelo Bernard Leputla (202403700)
GitHub	TumeloLeputla2006	**Due**	Sun 11 Oct 2026, 23:59

1 Your role in M2

- Design the **five-stage pipeline**, the **pipeline-register data model** and the **control unit**; design the **hazard unit** (forwarding, load-use stall, branch flush).
- Own `src/pipeline_model.js`: data structures, `controlSignals()`, `forwardSelect()`, `loadUseStall()`, `branchTaken()`. They are unit-tested now and wired into the stages in M3 (pipeline) and M4 (visual hazards).

2 Block diagram



Separate instruction memory (IF) and data memory (MEM). Solid arrows: data; dashed: stall control.

Figure 1: LIMO pipeline with forwarding (green, blue), branch/jump redirect (red) and load-use stall (orange).

3 Stage by stage

Stage	Inputs	Work	Outputs (into the next pipeline register)
IF	PC	instr = imem[PC]; next PC = PC + 4, or the target if a taken branch/jump is signalled by EX, or unchanged if stalled	IF/ID: pc, instr
ID	IF/ID, register file	decode; rs1Val, rs2Val read (after WB has written this cycle); imm = sign-extended immediate; control signals; load-use check against ID/EX	ID/EX: operands, imm, rd, f3, control
EX	ID/EX, EX/MEM, MEM/WB	ForwardA/B choose the operands; ALU(aluCtrl) on (A, aluSrc ? imm : B); branch comparator on (A, B) with funct3; target = pc + imm (branch/tlk/tl) or (A + imm) with bit 0 cleared (tlkr); jumps put PC+4 in aluOut; taken = branch & compare, or jump	EX/MEM: aluOut, storeData = forwarded B, rd, control; PC redirect + flush
MEM	EX/MEM	memRead: memData = dmem word at aluOut; memWrite: store storeData at aluOut	MEM/WB: aluOut, memData, rd, control
WB	MEM/WB	if regWrite and rd ≠ 0: regs[rd] = memToReg ? memData : aluOut	register file

4 Pipeline-register data model

Register	Fields held (in `src/pipeline_model.js`)	Written by	Read by
IF/ID	valid, pc, instr (32 b); pc4 = pc + 4 (derived, kept for display)	IF	ID
ID/EX	valid, pc, instr, rs1, rs2, rd (5 b each), rs1Val, rs2Val, imm (32 b, sign-extended), f3, and all control signals: regWrite aluSrc aluCtrl memRead memWrite memToReg branch jump jumpReg useRs1 useRs2	ID	EX, hazard unit
EX/MEM	valid, pc, instr, rd, aluOut (32 b: ALU result, or PC+4 for jumps), storeData (32 b, forwarded rs2), taken, target, control: regWrite memRead memWrite memToReg	EX	MEM, forwarding unit
MEM/WB	valid, pc, instr, rd, aluOut, memData (32 b), control: regWrite memToReg	MEM	WB, forwarding unit
Architectural state	pc; regs[32] (R0 always 0); imem (instruction words); dmem (4096 bytes, little-endian); cycle; stats {instructions, stalls, flushes}	-	-

5 Control unit

Class (instructions)	RegW	ALUSrc	ALUCtl	MemR	MemW	M2R	Br	Jmp	JReg	rs1	rs2
R-type (kpn tls le kp kpe stts stto)	1	0	by funct	0	0	0	0	0	0	1	1
I-type ALU (kpnh leh kph kpeh)	1	1	by funct3	0	0	0	0	0	0	1	0
Load (tsl)	1	1	ADD	1	0	1	0	0	0	1	0
Store (bl)	0	1	ADD	0	1	0	0	0	0	1	1
Branch (khl khsl khn khk)	0	0	-	0	0	0	1	0	0	1	1
Jump and link (tlk, tl)	1	0	-	0	0	0	0	1	0	0	0
Jump register (tlkr)	1	1	ADD	0	0	0	0	1	1	1	0

ALUCtrl	AND	OR	ADD	XOR	SLL	SRL	SUB
code (4 bits)	0000	0001	0010	0011	0100	0101	0110
used by	le, leh	kp, kph	kpn, kpnh, tsl, bl, tlkr	kpe, kpeh	stts	stto	tls

Key: RegW = RegWrite, ALUCtl = ALUCtrl, MemR = MemRead, MemW = MemWrite, M2R = MemToReg, Br = Branch, Jmp = Jump, JReg = JumpReg; rs1 / rs2 = 1 when the instruction really reads that source register (used by the hazard unit). "-" = not used. The rs1/rs2 columns tell the hazard unit which source registers matter; for kpnh the bits at the rs2 position are immediate bits and must not cause a stall (tested in group E). Control meaning of each signal (one line each, for the Control view) is stored in `SIGNAL_MEANING`.

6 Hazard unit rules

Hazard	Detected	Action	Cost
RAW, result 1 instruction earlier	EX/MEM.regWrite, EX/MEM.rd ≠ 0, EX/MEM.rd = source	ForwardA/B = 2 (take EX/MEM.aluOut)	0 cycles
RAW, result 2 instructions earlier	MEM/WB.regWrite, MEM/WB.rd ≠ 0, MEM/WB.rd = source (and EX/MEM does not match)	ForwardA/B = 1 (take MEM/WB value)	0 cycles
RAW, result 3 instructions earlier	none needed	register file writes first and ID reads second in the same cycle	0 cycles
Load-use	in ID: ID/EX.memRead, ID/EX.rd ≠ 0, rd = a source that the ID instruction reads	PCWrite = 0, IF/ID hold, bubble into ID/EX; next cycle MEM/WB forwarding	1 cycle
Control (taken branch/jump)	in EX: branch & compare true, or jump	PC = target; IF/ID and ID/EX become bubbles	2 cycles
Forwarding OFF (M4 switch)	-	stall in ID until the producer has written the register file (2 or 1 bubbles)	shown in M4

Flush and stall never happen in the same cycle: a load-use stall needs a load in EX, a flush needs a branch/jump in EX. A stored value (bl rs2) is forwarded like any operand.

7 Cycle diagrams

RAW resolved by forwarding (green = forwarded, no stall):

Instruction	1	2	3	4	5	6	7
`kpnh R5, R0, 3`	IF	ID	EX	MEM	WB		
`kpn R6, R5, R5`		IF	ID	EX fwd A,B	MEM	WB	
`kpn R7, R6, R5`			IF	ID	EX fwd A,B	MEM	WB

Load-use stall (orange = stall, green = forwarded):

Instruction	1	2	3	4	5	6	7	8
`tsl R6, 0(R11)`	IF	ID	EX	MEM	WB			
`kpn R7, R5, R6`		IF	ID	ID stall	EX fwd B	MEM	WB	
`leh R8, R7, 15`			IF	IF held	ID	EX fwd A	MEM	WB

Taken branch flushes two instructions (red):

Instruction	1	2	3	4	5	6	7	8
`khk R5, R7, DONE (taken)`	IF	ID	EX: taken	MEM	WB			
`wrong-path instruction 1`		IF	ID	flushed				
`wrong-path instruction 2`			IF	flushed				
`instruction at the target`				IF	ID	EX	MEM	WB

8 How the simulator will step one clock cycle (M3 plan)

1. Copy the current state (all pipeline registers, PC, registers).
2. Compute every stage from the *old* copy, in this order: WB (register write), MEM, EX (forwarding, ALU, compare), ID (decode, read registers, hazard check), IF.
3. Decide: if EX says taken → new PC = target and IF/ID, ID/EX become bubbles; else if load-use → keep PC and IF/ID, bubble into ID/EX; else normal advance.
4. Commit all new pipeline registers at once; cycle = cycle + 1; update statistics (stalls, flushes, instructions retired).
5. Stop when PC is past the program and all four pipeline registers are bubbles. **cycles = number of clock cycles executed** (matches the formula in the test report).

9 Defence preparation

Likely defence question	Answer you should be able to give
Why does a load followed by a dependent instruction stall under full forwarding?	The loaded value exists only after MEM, but the next instruction needs it at the start of its EX in the same cycle: one bubble, then forward from MEM/WB.
Which control signals are 1 for `tsl`?	RegWrite, ALUSrc, MemRead, MemToReg (ALUCtrl = ADD).
What does ForwardA = 2 mean?	Operand A of the ALU comes from EX/MEM.aluOut, the result of the instruction one ahead.
Why resolve branches in EX?	Reuses the ALU/forwarding hardware; costs two flushed instructions. ID resolution needs extra comparators and stalls and gave 65 vs 64 cycles for Program 1.
Why is R0 never forwarded?	R0 is hard-wired to 0; a write to it is ignored, so forwarding it would give a wrong non-zero value.
Which registers are architectural?	PC, R0–R31 and memory. IR, pipeline registers, control signals and bubbles are microarchitecture and must not change a program's result.